

FinalProject2

June 12, 2025

1 Final Project 2 (Using NIRCAM image of NGC 3324)

```
[1]: import numpy as np
import sep
```

```
[2]: import astropy.io.fits as fits
import matplotlib.pyplot as plt
from matplotlib import rcParams

%matplotlib inline

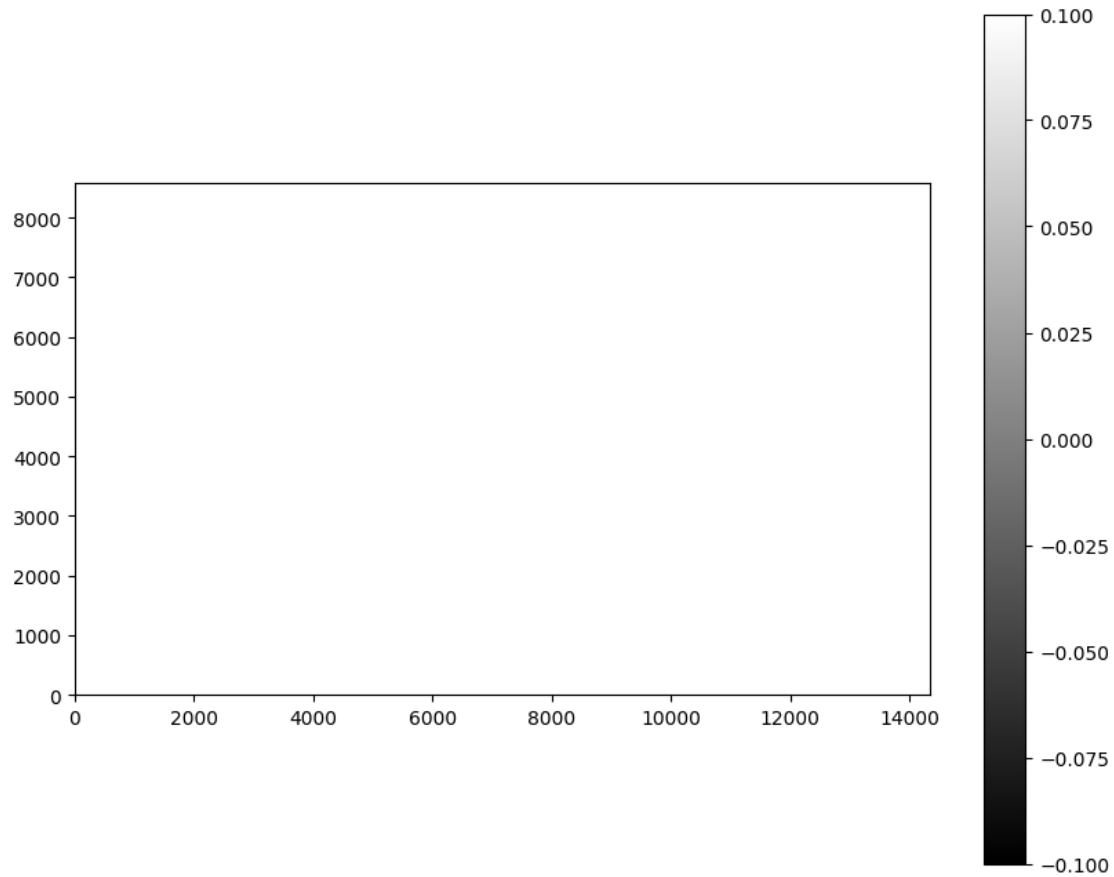
rcParams['figure.figsize'] = [10., 8.]
```

1.1 Read image into standard 2-d numpy array

```
[3]: fits_image_filename = 'jw02731-o001_t017_nircam_clear-f090w_i2d.fits'
with fits.open(fits_image_filename) as hdul: # open a FITS file
    data = hdul[1].data
```

1.2 Show the image and save as image_plot.png

```
[4]: m, s = np.mean(data), np.std(data)
plt.imshow(data, interpolation='nearest', cmap='gray', vmin=m-s, vmax=m+s,
           origin='lower')
plt.colorbar();
```



1.3 Measure a spatially varying background on the image

```
[5]: data = data.astype(data.dtype.newbyteorder('='))  
     bkg = sep.Background(data)
```

1.4 Get a “global” mean and noise of the image background:

```
[6]: print(bkg.globalback)  
     print(bkg.globalrms)
```

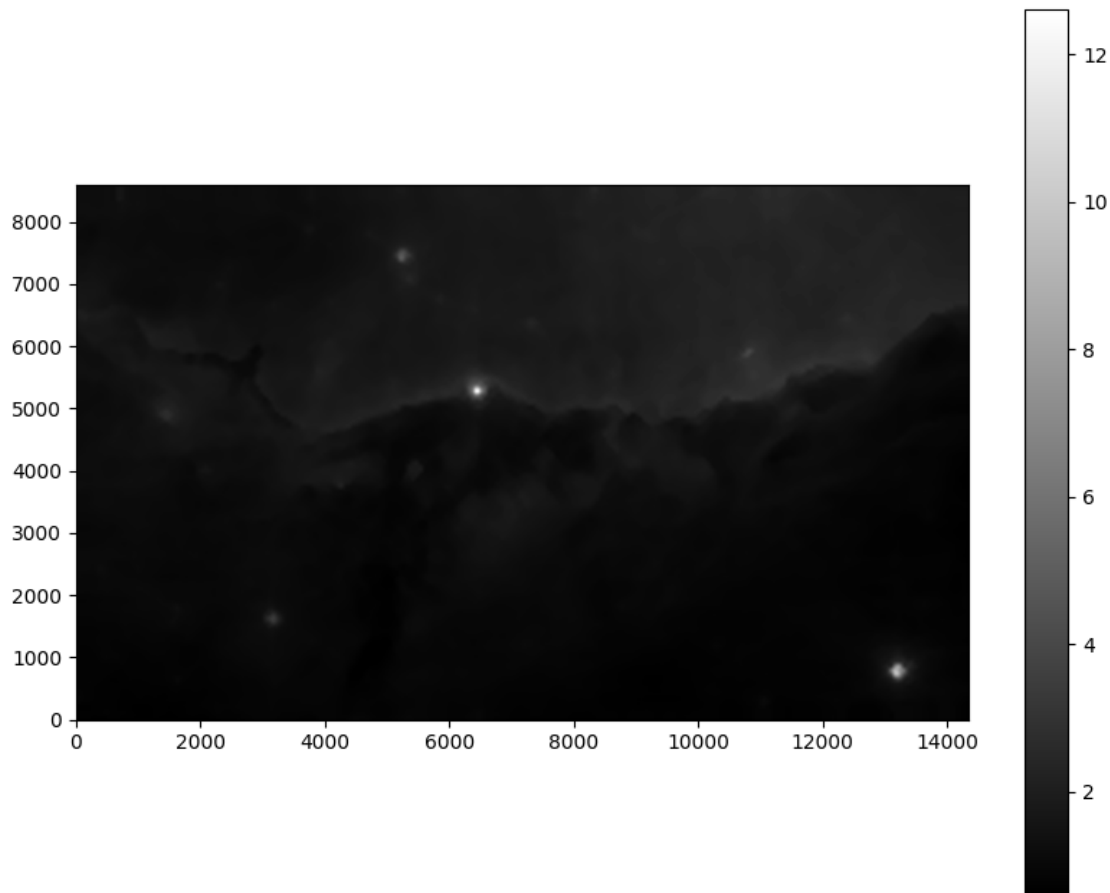
```
1.1997147798538208  
0.06582547724246979
```

1.5 Evaluate background as 2-d array, same size as original image

```
[7]: bkg_image = bkg.back()
```

1.6 Show the background and save as background_plot.png

```
[8]: plt.imshow(bkg_image, interpolation='nearest', cmap='gray', origin='lower')  
plt.colorbar();  
  
plt.savefig('background_plot.png')
```

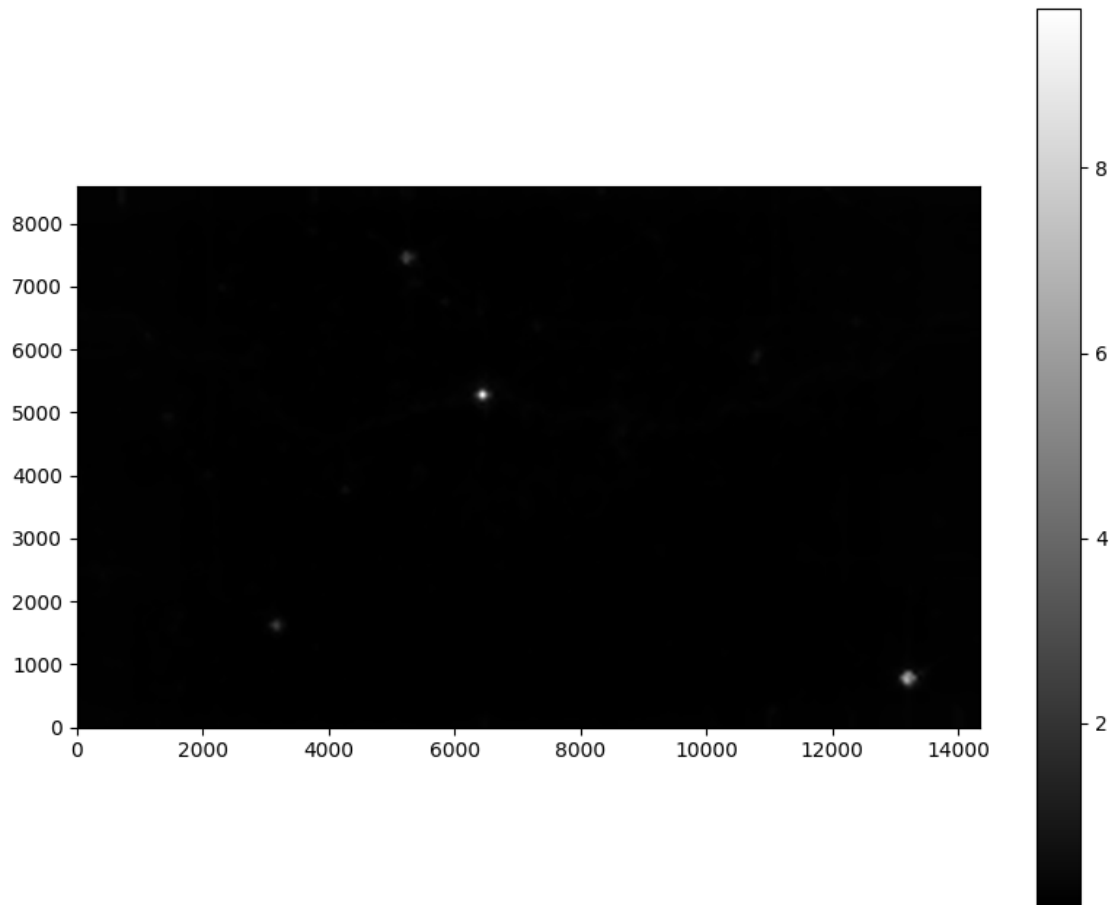


1.7 Evaluate the background noise as 2-d array, same size as original image

```
[9]: bkg_rms = bkg.rms()
```

1.8 Show the background noise and save the figure as background_noise.png

```
[10]: plt.imshow(bkg_rms, interpolation='nearest', cmap='gray', origin='lower')  
plt.colorbar();  
  
plt.savefig('background_noise.png')
```



1.9 Subtract the background

```
[11]: data_sub = data - bkg
```

1.10 Detect objects and set the detection threshold to be a constant value of 1.5 standard deviations

```
[12]: objects = sep.extract(data_sub, 1.5, err=bkg.globalrms)
```

```
-----
Exception                                Traceback (most recent call last)
Cell In[12], line 1
----> 1 objects = sep.extract(data_sub, 1.5, err=bkg.globalrms)

File sep.pyx:787, in sep.extract()

File sep.pyx:299, in sep._assert_ok()
```

```
Exception: internal pixel buffer full: The limit of 300000 active object pixels,
↳ over the detection threshold was reached. Check that the image is background-
↳ subtracted and the detection threshold is not too low. If you need to increase
↳ the limit, use set_extract_pixstack.
```

1.11 Print how many objects were detected

```
[13]: len(objects)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[13], line 1
----> 1 len(objects)

NameError: name 'objects' is not defined
```

1.12 Over-plot the object coordinates with some basic shape parameters on the image and then save figure as image_coordinates.png

```
[14]: from matplotlib.patches import Ellipse

# plot background-subtracted image
fig, ax = plt.subplots()
m, s = np.mean(data_sub), np.std(data_sub)
im = ax.imshow(data_sub, interpolation='nearest', cmap='gray',
               vmin=m-s, vmax=m+s, origin='lower')

# plot an ellipse for each object
for i in range(len(objects)):
    e = Ellipse(xy=(objects['x'][i], objects['y'][i]),
                width=6*objects['a'][i],
                height=6*objects['b'][i],
                angle=objects['theta'][i] * 180. / np.pi)
    e.set_facecolor('none')
    e.set_edgecolor('red')
    ax.add_artist(e)

plt.savefig('image_coordinates.png')
```

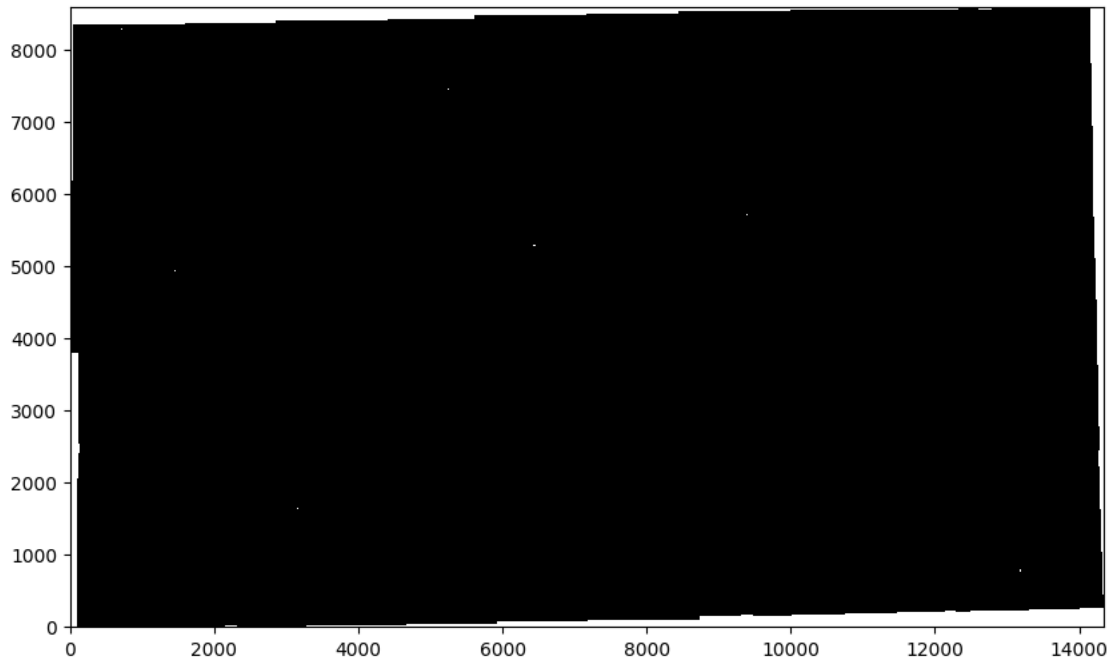
```
-----
NameError                                Traceback (most recent call last)
Cell In[14], line 10
      6 im = ax.imshow(data_sub, interpolation='nearest', cmap='gray',
      7                 vmin=m-s, vmax=m+s, origin='lower')
      9 # plot an ellipse for each object
--> 10 for i in range(len(objects)):
```

```

11     e = Ellipse(xy=(objects['x'][i], objects['y'][i]),
12                  width=6*objects['a'][i],
13                  height=6*objects['b'][i],
14                  angle=objects['theta'][i] * 180. / np.pi)
15     e.set_facecolor('none')

```

NameError: name 'objects' is not defined



1.13 List the available object fields

```
[15]: objects.dtype.names
```

```

-----
NameError                                Traceback (most recent call last)
Cell In[15], line 1
----> 1 objects.dtype.names

NameError: name 'objects' is not defined

```

1.14 Perform simple circular aperture photometry with a 3 pixel radius at the locations of the objects

```
[16]: flux, fluxerr, flag = sep.sum_circle(data_sub, objects['x'], objects['y'], 3.0,
      ↪err=bkg.globalrms, gain=1.0)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[16], line 1
----> 1 flux, fluxerr, flag = sep.sum_circle(data_sub, objects['x'],
      ↪objects['y'], 3.0, err=bkg.globalrms, gain=1.0)

NameError: name 'objects' is not defined
```

1.15 Show the first 10 objects results:

```
[17]: for i in range(10):
      print("object {:d}: flux = {:f} +/- {:f}".format(i, flux[i], fluxerr[i]))
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[17], line 2
      1 for i in range(10):
----> 2     print("object {:d}: flux = {:f} +/- {:f}".format(i, flux[i],
      ↪fluxerr[i]))

NameError: name 'flux' is not defined
```

2 Step 7

```
[18]: print("Mean of fluxes:", np.mean(flux))
      print("Median of fluxes:", np.median(flux))
      print("Standard deviation of fluxes:", np.std(flux))
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[18], line 1
----> 1 print("Mean of fluxes:", np.mean(flux))
      2 print("Median of fluxes:", np.median(flux))
      3 print("Standard deviation of fluxes:", np.std(flux))

NameError: name 'flux' is not defined
```

3 Step 8

```
[19]: # -----
# 0. Copy all this script into one cell, between steps 7 and 8 of your final
#    ↪project
#    You will need to install the "reproject" package with pip
#    This script will "reproject" all images onto the same shape using one
#    ↪image as a reference
# -----

from astropy.io import fits
from astropy.wcs import WCS
from reproject import reproject_interp

# -----
# 1. Load your reference image
#    This will define the WCS and shape for all others
# -----
file_reference = 'jw02731-o001_t017_nircam_clear-f090w_i2d.fits'
with fits.open(file_reference) as hdulist:
    # For JWST i2d images, 'SCI' is commonly the science extension
    ref_header = hdulist['SCI'].header
    ref_data = hdulist['SCI'].data

# Create a WCS object from the reference
ref_wcs = WCS(ref_header)

# This shape will be used for all reprojected images
output_shape = ref_data.shape

# -----
# 2. Reproject the other images
# -----
fname1 = jw02731-o001_t017_nircam_clear-f187n_i2d.fits
fname2 = jw02731-o001_t017_nircam_clear-f200w_i2d.fits
fname3 = jw02731-o001_t017_nircam_clear-f335m_i2d.fits
fname4 = jw02731-o001_t017_nircam_clear-f444w_i2d.fits
fname5 = jw02731-o001_t017_nircam_f444w-f470n_i2d.fits

other_filenames = [
    fname1,
    fname2,
    fname3,
    fname4,
    fname5,
]
```



```

# Reproject all other files according to the reference
for fname in other_filenames:
    with fits.open(fname) as hdulist:
        data = hdulist['SCI'].data
        header = hdulist['SCI'].header
        wcs_in = WCS(header)

    # Reproject this image onto the reference WCS
    # reproject_interp returns (reprojected_data, footprint)
    reprojected_data, footprint = reproject_interp(
        (data, wcs_in),
        ref_wcs,
        shape_out=output_shape
    )

    # Optionally, update the header to match the reference WCS
    # so that the new FITS is self-consistent
    new_header = ref_header.copy()

    # Save the new file
    out_name = fname.replace('.fits', '_reproj.fits')
    hdu = fits.PrimaryHDU(reprojected_data, header=new_header)
    hdu.writeto(out_name, overwrite=True)
    print(f"Reprojected {fname} -> {out_name}")

```

WARNING: FITSFixedWarning: 'datfix' made the change 'Set DATE-BEG to '2022-06-03T09:39:52.572' from MJD-BEG.
Set DATE-AVG to '2022-06-03T12:17:28.687' from MJD-AVG.
Set DATE-END to '2022-06-03T14:54:43.830' from MJD-END'. [astropy.wcs.wcs]
WARNING: FITSFixedWarning: 'obsfix' made the change 'Set OBSGEO-L to -82.733698 from OBSGEO-[XYZ].
Set OBSGEO-B to -31.567170 from OBSGEO-[XYZ].
Set OBSGEO-H to 1654335564.228 from OBSGEO-[XYZ]'. [astropy.wcs.wcs]

```

-----
NameError                                Traceback (most recent call last)
Cell In[19], line 30
    25 output_shape = ref_data.shape
    27 # -----
    28 # 2. Reproject the other images
    29 # -----
--> 30 fname1 = jw02731-o001_t017_nircam_clear-f187n_i2d.fits
    31 fname2 = jw02731-o001_t017_nircam_clear-f200w_i2d.fits
    32 fname3 = jw02731-o001_t017_nircam_clear-f335m_i2d.fits

NameError: name 'jw02731' is not defined

```